

گزارش کلی پروژه OMERTAOS / شاخه CAPO

تاریخ تهیه: ۱۶ اوت ۲۰۲۶

شاخه محلی: codex/capo-r4-validation

کامیت فعلی محلی: 02fbce8dd9c21dfea59a5a185e5e9bab6b4e04a6

کامیت مرجع CAPO شناسایی شده از GitHub API: 863e00c6398bdd03a78140e9607c032a8b1025d3

مورد	وضعیت
شاخه محلی	codex/capo-r4-validation
کامیت فعلی	02fbce8dd9c21dfea59a5a185e5e9bab6b4e04a6
کامیت مرجع CAPO	863e00c6398bdd03a78140e9607c032a8b1025d3
محدودیت محیط	اجرای sequential، حدود ۸ GiB RAM، Linux host بدون merge یا push

خلاصه مدیریتی

OMERTAOS در شاخه CAPO از حالت نمونه پژوهشی به سمت یک زیرساخت Runtime قابل بازتولید حرکت کرده است. در این دور کاری، ابتدا وضعیت واقعی مخزن و ادعاهای مستند بررسی شد، سپس بدون کار مستقیم روی CAPO یک شاخه امن محلی ساخته شد، گیت‌های موجود به ترتیب اجرا شدند، شکست‌های بازتولیدپذیر تشخیص داده شدند و چند تعمیر حداقلی برای بازگرداندن مسیرهای اصلی، build، Docker Quickstart، test و اعتبارسنجی backend انجام شد.

وضعیت فعلی پروژه را باید «نمونه پیاده‌سازی شده با شواهد محلی قابل اجرا» دانست، نه محصول production-ready. معماری Console -> Gateway -> Control -> Runtime در تست‌های معماری و بخشی از اجرای Docker backend قابل مشاهده است، اما اجرای موفق task روی Runtime، ایزولاسیون کامل لینوکسی، مقیاس‌پذیری چند native/systemd acceptance، worker و benchmark علمی هنوز اثبات نشده‌اند.

مهم‌ترین پیشرفت جدید، اضافه شدن یک زیرسامانه حداقلی در Control برای ثبت Runtime node، heartbeat، تشخیص وضعیت eligibility، node بر اساس tenant/capability/capacity و schedulerهای round-robin و least-loaded است. این بخش شواهد E2 دارد، اما هنوز distributed membership، consensus، failover یا اجرای واقعی workload روی Runtime را ثابت نمی‌کند.

هدف پروژه

هدف OMERTAOS ساخت یک مسیر اجرایی توزیع شده و قابل ممیزی برای اجرای taskهاست که در آن Console سطح کاربری، Gateway مرز ورودی و Control، transport، مالک orchestration و policy، و Runtime مالک اجرای محدود و ایزوله شده باشد. برای manuscript مربوط به FGCS، ارزش اصلی پروژه زمانی قابل دفاع است که ادعاها با تست، build، لاگ، پذیرش Docker/native و benchmark قابل بازتولید پشتیبانی شوند.

در وضعیت فعلی، پروژه به درستی از ادعاهای بزرگ‌تر از شواهد پرهیز می‌کند. دفتر ادعاها سطح شواهد را جدا کرده و صریحاً مشخص می‌کند کدام بخش‌ها E0، E1، E2، E3 هستند. این کار برای مسیر پژوهشی مهم است، چون اجازه می‌دهد مقاله به جای ادعای production readiness، روی مسیر قابل بازتولید و مرزهای اثبات شده تمرکز کند.

کارهای انجام شده تا این نقطه

در فازهای اولیه، محیط و مخزن بررسی شد. میزبان Ubuntu 22.04.5 LTS با kernel 6.8.0-124-generic، معماری x86_64، حدود ۷.۷ GiB RAM و ۲ GiB swap است. Docker Compose و Docker در دسترس هستند، ولی محدودیت حافظه جدی است و swap در زمان اعتبارسنجی‌ها چند بار پر گزارش شد. CPU میزبان Intel Core i7-5500U گزارش شد که با فرض اولیه «نسل هشتم» همخوان نیست.

به دلیل timeout در عملیات شبکه‌ای workspace، Git، از آرشیو source مربوط به CAPO بازسازی شد. شاخه امن codex/capo-r4-validation ایجاد شد و کارها بدون push، merge، force-push یا rewrite history انجام شدند. چند commit محلی برای checkpoint و تعمیرات ساخته شد.

در مسیر تعمیرات، مشکلات اصلی زیر برطرف شدند:

وابستگی‌های Docker برای Control در control/requirements.docker.txt قفل و تکمیل شدند. مسیر fail-closed حداقلی gRPC در Control اضافه شد تا Gateway بتواند به Control درخواست task بدهد و در نبود Runtime transport نسخه‌دار، خطای کنترل‌شده برگردد. ناسازگاری plugin‌های Fastify در Gateway با نسخه فعلی اصلاح شد. shutdown تمیز برای Gateway و Runtime بهبود پیدا کرد تا توقف containerها به خطای عملیاتی تبدیل نشود.

نصب قفل‌شده Console با pnpm-workspace.yaml اصلاح شد. Dockerfile Runtime برای build تک‌job تنظیم شد تا با محدودیت RAM میزبان سازگارتر باشد. اسناد evidence و گزارش CAPO R4 به‌روزرسانی شدند.

در فاز بعدی، زیرسامانه حداقلی Runtime scheduling در Control اضافه شد:

جدول‌ها و مدل‌های runtime_nodes، task_attempts و scheduling_decisions. endpointهای ثابت node و discovery در مسیر v1/runtime/nodes/. heartbeat با وضعیت‌های healthy، degraded، unreachable و draining. eligibility بر اساس capability، tenant، ظرفیت، freshness و حالت draining. schedulerهای round-robin و least-loaded. bounded retry، replay idempotent برای task/attempt تکراری، و ثبت audit evidence برای scheduling تصمیم بهبود helperهای Runtime برای رد کردن node id خالی و تولید resource report غیرخالی و bounded.

وضعیت اعتبارسنجی

گیت‌های قابل اجرای مخزن در این محیط تا سطح قابل قبولی بازتولید شدند. معماری ۶۸ تست پاس داشت. مجموعه کامل Python پس از فاز ۷ به ۱۸۷ passed و ۲ skipped رسید. تست‌های Gateway با ۶ تست پاس شدند. Console نصب قفل‌شده، Prisma generate، تست واحد و production build را پاس کرد. Runtime داخل Docker با image رسمی Rust و CARGO_BUILD_JOBS=1 تست شد و ۴ تست پاس کرد.

Docker Quickstart برای backend به صورت زنده اجرا شد: PostgreSQL، Redis، Qdrant، MinIO. یک Runtime worker، Gateway و Control health endpoint بالا آمدند. پاسخ سالم دادند. Runtime healthcheck از داخل شبکه Docker موفق بود. readiness برای PostgreSQL و Redis موفق بود. یک probe پایداری در PostgreSQL پس از restart عادی container باقی ماند. توقف stack با docker compose stop انجام شد و volume‌ها حذف نشدند.

مهم‌ترین محدودیت در پذیرش Docker این است که مسیر Control -> Gateway برای task submission به پاسخ 200 HTTP با status کاربردی ERROR و کد RUNTIME_TRANSPORT_UNAVAILABLE رسید. این یک failure کنترل‌شده و fail-closed است، نه اجرای موفق Runtime. بنابراین این نتیجه فقط جداسازی و اجرایی بودن transport تا Control را نشان می‌دهد و نباید به‌عنوان distributed execution گزارش شود.

سطح شواهد فعلی

معماری canonical request path و ownership مرزها سطح E1 دارد، چون تست‌های معماری آن را بررسی می‌کنند. Gateway و Control به‌عنوان سرویس‌های جدا با build/test و بخشی از acceptance سطح E2 محدود دارند. Runtime fail-closed در نبود sandbox backend سطح E2 دارد، اما موفقیت ایزولاسیون را ثابت نمی‌کند. زیرسامانه ثبت node و scheduler حداقلی در Control اکنون سطح E2 دارد.

ادعاهای زیر همچنان E0 هستند و نباید در گزارش علمی به‌عنوان نتیجه قطعی مطرح شوند:

- ایزولاسیون کامل Linux namespace, mount, seccomp و process.
- distributed membership کامل, consensus, federation یا leader election.
- مقیاس‌پذیری ۴ یا ۸ worker یا ۱۲۸ concurrency.
- latency/throughput بهتر از سیستم‌های دیگر.
- production readiness, security certification یا penetration testing مستقل.
- native Linux/systemd acceptance کامل.

وضعیت فایل‌ها و commit های محلی

کامیت‌های محلی مهم تا این نقطه:

```
c1d5827 chore: checkpoint CAPO snapshot 863e00c
docs(capo): record r4 validation evidence 7528516
daf6793 docs(native): add acceptance planning evidence
fix(capo): repair quickstart validation blockers 0906104
02fbce8 feat(control): add minimal runtime node scheduler
```

هیچ push, merge, release یا تغییر GitHub Issues انجام نشده است. کارها در شاخه محلی codex/capo-r4-validation باقی مانده‌اند.

وضعیت آینده نزدیک

اولویت بعدی باید بستن شکاف بین Control و Runtime باشد. الان Gateway می‌تواند به Control برسد، ولی Control هنوز execution transport نسخه‌دار و موفق به Runtime ندارد. کوچک‌ترین milestone آینده باید یک قرارداد execution روشن، تست‌های منفی/مثبت، idempotency در مسیر واقعی، و audit کامل برای درخواست، انتخاب worker، ارسال execution، پاسخ Runtime و خطایابی باشد.

هم‌زمان باید path live Console روی Docker با محافظ حافظه بررسی شود. اگر container کامل Console روی این host سنگین باشد، ابتدا health/backend و سپس build/test frontend جداگانه باقی بماند و گزارش نیز آن را partial acceptance بنامد. اجرای browser-level end-to-end فقط وقتی باید ادعا شود که واقعا Runtime -> Control -> Gateway -> Console یا fail-closed boundary قابل مشاهده باشد.

برای native Linux نیز باید قبل از هر اقدامی approval گرفته شود، چون ساخت service user، نوشتن etc/omertaos/unit های systemd و start target نیازمند sudo هستند. مسیر درست، اجرای reversible, smoke test، نصب env file ها با secret خارج از Git، نصب preflight read-only، آماده‌سازی rollback و reboot recovery، backup، restore dry-run، update.

نقشه راه پیشنهادی

کوتاه‌مدت:

- تکمیل Control-to-Runtime execution transport با قرارداد نسخه‌دار.
- اضافه کردن تست‌های regression برای execution success, timeout, unreachable worker و retry bounded.

اجرای Docker acceptance با یک Runtime worker و سپس حداکثر دو worker.
اجرای Console live health در صورت کفایت RAM.
آماده کردن ابزار یا container مناسب برای cargo fmt --check و security scan های مسدودشده.

میان مدت:

سخت سازی Runtime isolation با evidence منفی و escape tests روی Linux host سازگار.
تکمیل audit trail end-to-end از request تا scheduling decision و execution result.
پیاده سازی drain عملیاتی برای worker ها و recovery پس از restart.
اضافه کردن migration و rollback تست شده برای جدول های scheduling.
اجرای native N1 acceptance فقط پس از approval و روی محیط قابل بازگشت.

بلند مدت:

طراحی و اجرای benchmark سبک با concurrency های ۱، ۴، ۸ و ۱۶، worker های ۱ و ۲،
tenant های ۱، ۵ و ۱۰، و latency mock کنترل شده.
ثبت metadata، raw CSV/JSON، محیط، confidence interval، commit SHA، artifact های
sanitized.
آماده سازی بسته evidence برای manuscript شامل claim ledger، reproducibility
threat model و commands، acceptance report، limitations.
بررسی طراحی بزرگ تر distributed runtime فقط بعد از اثبات execution path کوچک.

ریسک ها و محدودیت ها

ریسک اصلی پروژه، جلو افتادن متن ادعاها از شواهد اجرایی است. تا وقتی execution واقعی Runtime isolation، موفق، native acceptance و benchmark کنترل شده اجرا نشده اند، زبان مقاله باید محتاط بماند. ریسک دوم، محدودیت سخت افزاری host است؛ ۸ GiB RAM برای build های هم زمان و stack کامل مناسب نیست و همه validation ها باید sequential و با محافظ حافظه اجرا شوند.
ریسک سوم، تفاوت محیط محلی با محیط پژوهشی هدف است. نبود Rust toolchain host، نبود trivy و cargo audit، و نیاز به bundled Node برای Gateway نشان می دهد reproducibility باید دقیق و وابسته به نسخه ابزارها ثبت شود. ریسک چهارم، امنیت secrets و env file ها است؛ هیچ secret نباید در گزارش، لاگ یا commit وارد شود.

پیشنهاد Issue های بعدی

۱. پیاده سازی و اعتبارسنجی Control-to-Runtime execution transport نسخه دار با fail-closed error model.
۲. افزودن Docker live acceptance برای مسیر task واقعی با یک Runtime worker و sanitized artifact های.
۳. آماده سازی Rust formatting/security toolchain قابل بازتولید بدون نصب system-wide.
۴. اجرای Console live smoke test با محدودیت RAM و ثبت partial/full acceptance.
۵. تکمیل native N1 acceptance plan با rollback، approval، restore dry-run و.

جمع بندی

پروژه OMERTAOS/CAPO اکنون نسبت به حالت اولیه قابل بازتولیدتر و صادق تر شده است: build ها و تست های اصلی به وضعیت پاس رسیده اند، Docker Quickstart backend در محدوده محدود اجرا شده، persistence پایه بررسی شده و یک scheduler حداقلی در Control اضافه شده است. با این حال، نقطه کلیدی بعدی همچنان اجرای واقعی و ممیزی شده Runtime است. مسیر درست از اینجا، کوچک نگه داشتن milestone ها، ثبت دقیق شواهد، و جلوگیری از ادعاهای بزرگ تر از نتیجه های اجرایی است.